

Trabajo de Arquitectura de Software

La Gran Liga de Apuestas de Medianos EIA

Contexto

Hace varios años, en una zona misteriosa ubicada detrás de los laboratorios de la Universidad EIA, vivía pacíficamente una pequeña comunidad de **medianos**.

Los medianos llevaban una vida tranquila: comían siete veces al día, evitaban las clases de las 6:00 a. m. y discutían constantemente sobre cuál cafetería de la Universidad vendía el mejor almuerzo.

Todo cambió cuando apareció **Dyllan, el gigante de Salchipapa Maker**.

Dyllan, obsesionado con encontrar nuevos participantes para sus experimentos deportivos, capturó a los medianos y creó la primera:

Liga Profesional de Carreras de Medianos de la Universidad EIA.

Las carreras rápidamente se volvieron populares.

Profesores, estudiantes, administrativos y misteriosos personajes externos comenzaron a apostar créditos ficticios llamados **EIA Coins** sobre cuál mediano ganaría cada carrera.

Nota: los EIA Coins son créditos ficticios de este enunciado: no tienen valor monetario, no se compran ni se canjean. El ejercicio no promueve apuestas con dinero real.

Lo que comenzó como una hoja de cálculo manejada por Dyllan terminó convirtiéndose en un negocio inesperadamente grande.

Ahora existen miles de usuarios, cientos de carreras, estadísticas históricas, apuestas en tiempo real y transmisiones en vivo.

Dyllan necesita una plataforma tecnológica profesional.

El problema es que Dyllan sabe hacer salchipapas, pero no sabe diseñar sistemas distribuidos.

Su equipo deberá hacerlo por él.

El reto



La empresa **Salchipapa Maker Racing S.A.S.** los ha contratado como arquitectos de software.

Su misión será **diseñar la arquitectura tecnológica de la plataforma de apuestas de carreras de medianos**.

No deberán implementar la solución.

El objetivo del trabajo es analizar el problema y tomar decisiones arquitectónicas justificadas.

Deberán decidir, entre otras cosas:

- si la solución debe construirse como un **monolito**;
- si debe utilizar una arquitectura de **microservicios**;
- si sería conveniente utilizar una arquitectura **híbrida**;
- qué operaciones deberían comunicarse de manera **síncrona**;
- qué operaciones deberían funcionar mediante **eventos**;
- cuándo utilizar **RabbitMQ**;
- cuándo utilizar **Kafka**;
- cómo soportar grandes cantidades de información;
- cómo evitar operaciones duplicadas mediante **idempotencia**;
- cómo mantener consistentes los datos entre componentes;
- cómo mantener la plataforma segura;
- y cómo evitar que la infraestructura cueste más que todos los medianos juntos.

Funcionamiento del negocio

Cada semana se realizan diferentes carreras.

Una carrera tiene:

- identificador;
- fecha y hora;
- pista;
- distancia;
- medianos participantes;
- estado;
- resultados;
- ganador.

Los usuarios pueden consultar las carreras disponibles y realizar apuestas utilizando **EIA Coins**.

Una apuesta contiene:



- usuario;
- carrera;
- mediano seleccionado;
- cantidad apostada;
- fecha;
- identificador único de la operación.

Cuando termina una carrera, el sistema recibe los resultados oficiales y determina cuáles apuestas fueron ganadoras.

Posteriormente, se acreditan automáticamente las recompensas a las billeteras virtuales de los usuarios.

Reglas del negocio

Para que todos los equipos diseñen sobre los mismos supuestos, se fijan las siguientes reglas:

- **Modelo de premios (apuesta mutua):** todas las apuestas de una carrera forman una bolsa común. La casa retiene una comisión del 10 % y el resto se reparte entre las apuestas ganadoras, en proporción al monto apostado. Por tanto, el premio final solo se conoce cuando se cierran las apuestas.
- **Cuotas estimadas:** mientras la carrera está abierta, la plataforma muestra a los usuarios una cuota aproximada por mediano, que cambia a medida que llegan apuestas.
- **Cierre de apuestas:** las apuestas se cierran exactamente a la hora de inicio de la carrera, según el reloj del servidor. Una apuesta que llegue después del cierre debe rechazarse, aunque el usuario la haya enviado antes.
- **Varias apuestas:** un usuario puede realizar varias apuestas en la misma carrera, incluso sobre medianos distintos.
- **Cancelación:** si una carrera se cancela, se devuelve el 100 % de lo apostado a cada usuario.

Ciclo de vida de una carrera:

```
PROGRAMADA → ABIERTA → CERRADA → EN_CURSO → FINALIZADA → LIQUIDADA
```

Una carrera puede pasar a **CANCELADA** desde cualquier estado anterior a FINALIZADA.

Capacidades principales del sistema

La empresa ha identificado las siguientes capacidades.

1. Usuarios y autenticación



Debe permitir:

- registrar usuarios;
- iniciar sesión;
- consultar perfiles;
- administrar roles;
- bloquear usuarios;
- validar permisos.

Existen diferentes roles:

- apostador;
- operador de carrera;
- administrador;
- auditor.

La información sensible (contraseñas, datos personales y movimientos de la billetera) debe estar protegida.

---

2. Medianos

Debe almacenar la información de cada competidor.

Cada mediano tiene:

- nombre;
- edad;
- peso;
- historial de carreras;
- victorias;
- derrotas;
- velocidad promedio;
- estado actual.

Ejemplos de competidores:

- Frodito Turbo;
- Bilbo Rápido;
- Samwise Verstappen;
- Peregrin Hamilton;
- Merry Alonso.



Los estudiantes deberán decidir si esta capacidad merece tener un servicio independiente o podría formar parte de otro componente.

---

3. Carreras

Administra:

- creación de carreras;
- programación;
- participantes;
- horarios;
- pistas;
- inicio de carrera;
- finalización;
- resultados oficiales.

Cuando una carrera comienza o termina, otros componentes de la plataforma podrían necesitar conocer inmediatamente esta información.

El equipo deberá decidir **cómo comunicar estos cambios**.

---

4. Apuestas

Debe permitir realizar apuestas sobre las carreras disponibles.

Antes de aceptar una apuesta debe comprobarse:

- que el usuario exista;
- que la carrera esté abierta a apuestas (antes del cierre);
- que el mediano participe en la carrera;
- que el usuario tenga suficientes EIA Coins;
- que la apuesta no esté duplicada.

Durante carreras importantes se esperan **miles de apuestas por segundo**.

Por esta razón, esta funcionalidad será uno de los puntos más críticos de la arquitectura.



5. Billetera de EIA Coins

Cada usuario tendrá una billetera virtual.

Debe permitir:

- consultar saldo;
- descontar EIA Coins al realizar una apuesta;
- acreditar premios;
- consultar movimientos;
- realizar devoluciones cuando una carrera sea cancelada.

Nunca debe ser posible gastar dos veces los mismos EIA Coins.

---

6. Resultados y premios

Cuando termina una carrera debe recibirse el resultado oficial.

El sistema deberá:

1. identificar el ganador;
2. encontrar las apuestas ganadoras;
3. calcular los premios;
4. acreditar los EIA Coins;
5. registrar cada movimiento;
6. generar información para auditoría.

Una carrera popular podría acumular **más de un millón de apuestas**.

No sería conveniente intentar procesarlas todas mediante una única petición HTTP.

---

7. Notificaciones

Los usuarios pueden recibir mensajes cuando:

- una apuesta fue aceptada;
- comenzó una carrera;
- terminó una carrera;
- ganaron una apuesta;



- recibieron EIA Coins;
- una carrera fue cancelada.

Las notificaciones pueden enviarse mediante:

- correo electrónico;
- aplicación móvil;
- notificaciones push.

Una falla en el sistema de notificaciones **no debería impedir que una apuesta sea registrada**.

---

8. Estadísticas y analítica

Dyllan descubrió que puede vender estadísticas premium sobre los medianos.

El sistema deberá almacenar información como:

- número de apuestas;
- montos apostados;
- comportamiento de usuarios;
- carreras realizadas;
- rendimiento histórico de los medianos;
- probabilidades;
- tendencias;
- eventos generados por la plataforma.

Cada día podrían producirse **millones de registros**.

El sistema de analítica no necesariamente necesita conocer toda la información en tiempo real.

---

9. Auditoría y control de fraude

Toda operación que afecte saldos, apuestas o resultados debe quedar registrada.

El sistema deberá permitir:

- conservar un registro inalterable de apuestas, movimientos de billetera, premios y cambios de resultados;
- consultar ese registro con el rol auditor, que solo tiene permisos de lectura;



- detectar comportamientos sospechosos, como muchas cuentas apostando desde el mismo dispositivo o apuestas masivas segundos antes del cierre;
- reconstruir, para cualquier carrera, cómo se calcularon sus premios.

El problema de las cargas masivas

La popularidad de una carrera puede cambiar drásticamente.

Una carrera normal podría tener:

500 usuarios conectados.

Pero la final de la:

Copa Intersemestral Dyllan Salchipapa Maker

podría tener:

100.000 usuarios conectados simultáneamente.

Durante los últimos segundos antes del cierre de apuestas pueden llegar miles de solicitudes prácticamente al mismo tiempo.

Su arquitectura deberá explicar:

- qué componentes deberían poder escalar;
- cuáles no necesitan escalar de la misma manera;
- cómo absorberían picos de tráfico;
- cómo evitarían perder operaciones;
- qué sucedería si un servicio está temporalmente caído.

Supuestos de diseño

Para que las decisiones puedan evaluarse, se fijan los siguientes supuestos. Pueden ajustarlos si lo justifican explícitamente.

Aspecto	Supuesto
Usuarios concurrentes	500 en una carrera normal; 100.000 en la final de la Copa Intersemestral.
Pico de apuestas	10.000 apuestas por segundo durante los 30 segundos previos al cierre.



Aspecto	Supuesto
Respuesta al apostar	El usuario debe saber en menos de 1 segundo si su apuesta fue aceptada o rechazada.
Pérdida de datos	Ninguna apuesta aceptada ni ningún movimiento de billetera puede perderse.
Liquidación de premios	Todos los premios pagados en menos de 15 minutos después del resultado oficial.
Estadísticas	Pueden tener minutos de retraso; los reportes premium, hasta un día.
Auditoría	Los registros se conservan al menos 5 años.

Idempotencia: Dyllan no quiere pagar dos veces

Uno de los principales problemas detectados durante las pruebas fue el siguiente:

Un estudiante realiza una apuesta por **500 EIA Coins**.

La aplicación demora en responder.

El estudiante presiona nuevamente:

APOSTAR

y luego nuevamente:

APOSTAR

y una cuarta vez porque considera que presionar más fuerte hace que Internet funcione mejor.

El sistema recibe cuatro solicitudes.

Su diseño deberá garantizar que una misma operación lógica **no sea procesada cuatro veces**.

Deberán explicar cómo utilizarían mecanismos de **idempotencia**.

Por ejemplo, pueden considerar conceptos como:

- identificadores únicos de operación;
- idempotency keys;
- almacenamiento de operaciones procesadas;
- restricciones únicas en bases de datos;
- identificadores únicos de eventos.



No es necesario implementar estos mecanismos.

Sí es obligatorio explicar **dónde y por qué los utilizarían**.

Comunicación entre componentes

Uno de los principales objetivos del ejercicio será decidir **cómo deben comunicarse las diferentes partes del sistema**.

Podrán utilizar:

Comunicación síncrona

Por ejemplo:

```
HTTP / REST
gRPC
```

Puede tener sentido cuando un servicio necesita conocer inmediatamente la respuesta de otro.

Comunicación asíncrona

Podrán utilizar eventos o mensajes para operaciones que no necesitan completarse dentro de la misma petición.

Por ejemplo:

```
ApuestaRealizada
CarreraIniciada
CarreraFinalizada
ResultadoPublicado
PremioCalculado
PremioPagado
CarreraCancelada
```

Su equipo deberá decidir cuáles de estos eventos deberían existir.

RabbitMQ o Kafka



En caso de utilizar comunicación asíncrona deberán analizar cuál tecnología utilizar.

RabbitMQ

Puede considerarse para escenarios donde interesa especialmente:

- distribuir tareas;
- utilizar colas;
- procesar trabajos mediante diferentes consumidores;
- implementar reintentos;
- gestionar mensajes pendientes.

Ejemplo conceptual:

```
Carrera termina
      ↓
Cola de cálculo de premios
      ↓
Workers
      ↓
Procesamiento de apuestas
```

Kafka

Puede considerarse cuando existen:

- grandes volúmenes de eventos;
- múltiples consumidores interesados en la misma información;
- almacenamiento del historial de eventos;
- procesamiento de streams;
- analítica;
- necesidad de reproducir eventos posteriormente.

Ejemplo conceptual:

```
ApuestaRealizada
       ↓
     Kafka
       ↓
Analítica · Fraude · Estadísticas · Auditoría
```

Cada consumidor lee el mismo evento de forma independiente y a su propio ritmo.

No existe obligación de utilizar ambos.



El equipo deberá decidir:

RabbitMQ, Kafka, ambos o ninguno.

La decisión deberá estar argumentada técnicamente.

Seguridad

La plataforma maneja dinero virtual y operaciones que afectan el saldo de los usuarios.

Su diseño deberá considerar como mínimo:

- autenticación;
- autorización;
- roles;
- protección de APIs;
- cifrado de comunicaciones;
- manejo seguro de credenciales;
- protección de operaciones administrativas;
- limitación de tasa (rate limiting) y protección contra bots;
- registro de todas las acciones administrativas.

También deberán indicar cuáles componentes deberían ser públicos y cuáles deberían permanecer dentro de una red privada.

Costos

Dyllan estableció una regla:

“No quiero pagar una infraestructura de Netflix para una carrera de cinco medianos.”

Por esta razón, la arquitectura debe considerar el costo.

No siempre dividir una aplicación en veinte microservicios representa una mejor arquitectura.

El equipo deberá justificar:

- cuántos servicios necesita realmente;
- cuáles componentes pueden compartir infraestructura;



- cuáles componentes deben escalar independientemente;
- dónde sería razonable utilizar servicios administrados;
- cómo evitar infraestructura innecesaria.

Decisiones obligatorias

El equipo deberá responder explícitamente las siguientes preguntas.

Decisión 1 — Estilo arquitectónico

¿Construirían la solución utilizando:

- monolito;
- monolito modular;
- microservicios;
- arquitectura híbrida?

¿Por qué?

Mencionen al menos dos alternativas que descartaron y la razón por la cual las descartaron.

Decisión 2 — División de servicios

Definan los componentes o servicios principales de su arquitectura.

Como referencia existen capacidades relacionadas con:

- usuarios;
- medianos;
- carreras;
- apuestas;
- billetera;
- resultados;
- premios;
- notificaciones;
- estadísticas;
- auditoría.

No necesariamente cada capacidad debe convertirse en un microservicio.



El equipo deberá decidir cuáles separar y cuáles mantener juntas.

Decisión 3 — Comunicación síncrona

Indiquen cuáles componentes se comunicarían directamente mediante REST o gRPC.

Para cada comunicación completen una tabla como la siguiente:

Servicio origen	Servicio destino	Protocolo	Información solicitada	¿Por qué debe ser síncrona?

Indiquen también qué ocurre si el servicio destino no responde a tiempo (timeout, reintento o respuesta alternativa).

Decisión 4 — Comunicación asíncrona

Definan los principales eventos de la plataforma.

Pueden partir de los eventos sugeridos en la sección **Comunicación asíncrona** y agregar, fusionar o eliminar los que consideren necesarios. Usen un único idioma para nombrarlos (por ejemplo, ApuestaRealizada o BetPlaced, pero no ambos).

Para cada evento completen una tabla como la siguiente:

Evento	Productor	RabbitMQ / Kafka	Consumidor o consumidores	Clave de partición o routing key

Indiquen además qué información mínima contiene cada evento (por ejemplo, identificador del evento, fecha y datos del negocio).

Decisión 5 — RabbitMQ vs. Kafka



Expliquen:

- cuál utilizarían;
- dónde lo utilizarían;
- por qué;
- qué problema resuelve.

Si utilizan ambos, deberán justificar por qué mantener dos tecnologías diferentes representa una ventaja suficiente para asumir la complejidad adicional.

Decisión 6 — Idempotencia

Identifiquen **al menos tres operaciones** donde exista riesgo de duplicación.

Por ejemplo:

- crear una apuesta;
- cobrar una apuesta;
- pagar un premio;
- procesar un evento recibido más de una vez.

Expliquen cómo evitarían procesarlas múltiples veces.

Tengan en cuenta que, en su configuración habitual, RabbitMQ y Kafka pueden entregar el mismo mensaje más de una vez. Sus consumidores deben tolerarlo.

Decisión 7 — Grandes volúmenes

Expliquen cómo respondería su arquitectura cuando llegan:

10.000 apuestas por segundo durante los 30 segundos previos al cierre.

Deben identificar:

- posibles cuellos de botella;
- componentes que deberían escalar;
- uso de colas o streams;
- mecanismos para no perder información;
- cómo garantizarían que ninguna apuesta sea aceptada después del cierre;
- cómo evitarían que el saldo de un usuario quede negativo cuando envía varias apuestas en paralelo.



Decisión 8 — Fallos

Analicen al menos dos situaciones.

Caso A

El sistema de notificaciones está caído.

¿Los usuarios deberían poder seguir apostando?

Caso B

El servicio de billetera está caído.

¿Los usuarios deberían poder seguir apostando?

Expliquen por qué las respuestas podrían ser diferentes.

Caso C (opcional)

Diez minutos después de publicado el resultado oficial, un operador descubre que la foto de llegada fue mal interpretada y el ganador era otro mediano. Algunos premios ya fueron pagados.

¿Cómo corregirían el resultado sin perder la trazabilidad de lo ocurrido?

Decisión 9 — Datos y consistencia

Expliquen:

- qué tipo de base de datos usaría cada servicio o módulo y si la comparten o no;
- qué datos requieren consistencia fuerte y cuáles pueden ser eventualmente consistentes;
- cómo mantendrían coherentes la apuesta registrada y el descuento en la billetera si están en componentes distintos;
- dónde almacenarían el histórico de eventos y los datos para analítica.

Pueden considerar patrones como transacciones locales, sagas o transactional outbox, pero solo si realmente los necesitan.



Diseño de la arquitectura

El entregable debe contener un **diagrama general de arquitectura**.

Puede realizarse utilizando:

- Draw.io;
- Lucidchart;
- Excalidraw;
- Mermaid;
- PowerPoint;
- cualquier herramienta similar.

El diagrama debe mostrar claramente:

```
Cliente
   ↓
API / Gateway
   ↓
Servicios o módulos
   ↓
Bases de datos
   ↓
RabbitMQ / Kafka
   ↓
Consumidores
```

Deben diferenciar visualmente:

comunicación síncrona

de

comunicación asíncrona.

Además, deben incluir un **diagrama de secuencia** del flujo completo de una apuesta: desde que el usuario presiona APOSTAR hasta que recibe la confirmación, incluyendo el descuento en la billetera.

Diseño obligatorio de la mensajería

Además del diagrama general deberán diseñar el sistema de mensajería.



Si utilizan RabbitMQ deberán mostrar elementos como:

```
Exchange
Queue
Producer
Consumer
Routing
```

Si utilizan Kafka deberán mostrar:

```
Producer
Topic
Partitions
Consumer Groups
Consumers
```

No es necesario configurar ni ejecutar RabbitMQ o Kafka.

Únicamente deben diseñar la solución.

Entregable

Cada grupo deberá entregar un documento o presentación que contenga:

1. Arquitectura seleccionada y justificación (monolito, monolito modular, microservicios o híbrida), incluyendo las alternativas descartadas.
2. Identificación de servicios o módulos y sus responsabilidades.
3. Diagrama general de arquitectura.
4. Diagrama de secuencia del flujo de una apuesta.
5. Comunicaciones síncronas (tabla de la Decisión 3).
6. Eventos y comunicaciones asíncronas (tabla de la Decisión 4).
7. Diseño de RabbitMQ y/o Kafka.
8. Estrategia de idempotencia.
9. Datos y consistencia.
10. Manejo de cargas masivas.
11. Estrategia básica de seguridad, indicando qué componentes son públicos y cuáles privados.
12. Consideraciones de costo.
13. Manejo de fallos.
14. Registros de decisiones arquitectónicas (ADR) para las tres decisiones más importantes.
15. Respuesta a la pregunta final.
16. Conclusiones.

Cada ADR debe ocupar como máximo una página e incluir: contexto, opciones consideradas, decisión tomada y consecuencias (lo que se gana y lo que se sacrifica).



Condiciones y evaluación

- **Grupos:** máximo 3 estudiantes.
- **Fecha de entrega: Exposición 4 y 5 noviembre**.
- **Extensión:** presentación de máximo 20 diapositivas.
- **Uso de IA:** está permitido usar herramientas de IA generativa, pero todo lo entregado debe poder ser explicado y defendido por cualquier integrante del grupo.
- **Sustentación:** cada grupo presentará su arquitectura. Durante la sustentación se planteará un cambio de requisitos (por ejemplo, permitir apuestas en vivo mientras la carrera está en curso) y el equipo deberá explicar cómo lo absorbería su diseño.

Criterios de evaluación

Criterio	Peso
Justificación del estilo arquitectónico y de la división en servicios	15 %
Comunicación síncrona, asíncrona y diseño de la mensajería	20 %
Idempotencia, datos y consistencia	20 %
Escalabilidad y manejo de fallos	15 %
Seguridad y costos	10 %
Calidad de los diagramas, los ADR y el documento	10 %
Sustentación	10 %

Restricción importante

NO SE DEBE IMPLEMENTAR LA ARQUITECTURA.

Este trabajo busca evaluar la capacidad de:

analizar, diseñar, argumentar y tomar decisiones arquitectónicas.

No se calificará quién tenga más microservicios.

De hecho, crear:

```
42 microservicios
17 bases de datos
6 clusters de Kafka
4 clusters de RabbitMQ
```

para administrar cinco medianos probablemente sea una excelente manera de conseguir que Dyllan despida al equipo.

Pregunta final

Después de diseñar la solución deberán responder:

Si ustedes fueran los responsables técnicos de Salchipapa Maker Racing S.A.S., ¿qué arquitectura implementarían hoy y cómo permitirían que evolucionara si la plataforma creciera 100 veces?

La respuesta deberá estar sustentada en decisiones relacionadas con:

- complejidad;
- escalabilidad;
- resiliencia;
- consistencia;
- seguridad;
- costo;
- mantenibilidad.

Idea central del trabajo

Una buena arquitectura no es aquella que utiliza más tecnologías.

Una buena arquitectura es aquella que utiliza **la complejidad necesaria para resolver el problema sin agregar complejidad innecesaria**.

Y, sobre todo:

ningún mediano debe recibir dos premios porque alguien presionó cuatro veces el botón.